Eventify

Final Database Project

Team Name: G4C

Project: Eventify (Event Management System)

Project Overview

Eventify is a relational database system designed to support a local event management platform. The system manages event creation, venue scheduling, category classification, ticket purchasing, order tracking, and administrative oversight. The database is intended to demonstrate real-world business rules and relationships using MySQL tables, constraints, queries, views, stored procedures, and triggers.

Data Origin and Generation

All data used in the Eventify database is synthetic and was generated specifically for this course project. A Python-based data generation script created by the team, using a data-faker library (faker), produced realistic user profiles, venues, events, orders, and tickets at sufficient scale to support meaningful database queries. This generated data was exported into SQL INSERT statements and executed via dummy_data.sql. Additional controlled records are inserted in lifecycle demonstration scripts to provide predictable, repeatable workflows for testing triggers, procedures, and constraints.

Business Rules (Exhaustive):

User Management Rules

- Each user must have a unique email address.
- Users are assigned a role of Attendee, Organizer, or Admin.
- Users have an account status of Active or Suspended.
- Suspended users are not permitted to purchase tickets.
- A user may place multiple orders over time.
- An organizer is a user who is authorized to create events.

Event Management Rules

- Every event must be associated with exactly one organizer and one venue.
- Events may be assigned to one or more categories.
- Events must have a start time that occurs before the end time.
- Events have a lifecycle status of Draft, Published, Completed, or Canceled.
- Tickets may not be sold for Completed or Canceled events.
- Event capacity limits the number of tickets that may be sold.

Venue Rules

- Venues must define a non-negative seating capacity.
- Multiple events may be hosted at the same venue at different times.

Orders and Tickets

- Each order belongs to exactly one user.
- Each order may contain one or more tickets.
- Each ticket must be associated with one event and one order.
- Ticket prices must be non-negative values.
- Ticket status may be Reserved, Purchased, or Refunded.

Data Integrity Rules

- Referential integrity is enforced using foreign key constraints.
- Aggregated reporting values must reflect only valid completed transactions.

Database Implementation Summary

The Eventify database consists of normalized tables including users, venues, categories, events, event_category (junction table), orders, and tickets. Primary keys uniquely identify records, foreign keys enforce relationships, and enumerated fields validate domain constraints.

Normalization feedback from earlier milestones was incorporated to eliminate redundancy and correct foreign key placement.

Querying, Views, and Automation

The system includes multiple SQL queries demonstrating joins, aggregation, and subqueries. Views provide reusable access to published event information. Stored procedures summarize event sales and attendance metrics. Triggers enforce business rules by preventing ticket sales for canceled or completed events. Lifecycle demo scripts demonstrate attendee and organizer workflows from creation through cleanup.

Exported Schema and Submission Artifacts

The complete database schema, data population, triggers, views, and procedures were exported as a single SQL file using MySQL Workbench. Additional supporting SQL and Python files are included in the project repository to ensure the system can be recreated and evaluated consistently.